

AI Expense Analyzer

Course: CSE 4204-Mobile Computing Lab

Section: 8A

Team: CSE4204-8A-T04

Team Leader: Md. Shakinul Islam

Submitted To: Md. Riaz Mahmud, Assistant Professor, Dept. of CSE, NUBT Khulna

GitHub Repository: <https://github.com/shakinul-islam/AI-Expense-Analyzer>

Team Information

Name	Student ID	Technology	Responsibility
Md. Shakinul Islam (Lead)	11220120822	MongoDB	Database management & testing
Naim Sheikh	11210320634	Flutter	Frontend development & UI design
Ayon Banerjee	11220120820	Node.js + Express.js	Backend development
Afia Rahman	11220320870	Groq API	AI integration & documentation

1. Project Overview

Tech Stack: Flutter (frontend) · Node.js + Express.js (backend) · MongoDB (database) · Groq API (AI layer)

AI Expense Analyzer is a personal finance mobile application that helps users track income and expenses, manage budgets, and most importantly receive AI-generated financial insights, spending forecasts, and smart categorization based on their own transaction history.

1.1 Why AI Is Required

Manually tracking and interpreting personal spending is tedious, and most budgeting apps stop at showing raw numbers. This project integrates AI to go one step further: instead of just displaying transactions, the app analyzes spending patterns and proactively tells the user what they mean where they are overspending, how to save money, and what next month is likely to cost. This turns the app from a passive ledger into an active financial advisor, which is the real problem the AI integration solves.

2. Selected AI Platform

AI Platform: Groq API (LPU-based inference of open-weight Llama models)

AI Models Used: llama-3.3-70b-versatile and llama-3.1-8b-instant (see table below)

platform history: The SRS and System Design documents approved in Week 04-05 specified Google Gemini as the AI platform, and the architecture/activity diagrams below were drawn against that plan. During backend implementation the team migrated the AI Integration Service to Groq the Backend Progress Report (29/06/2026) lists the AI responsibility as "Groq API (Updated)" and services/aiService.js imports the groq-sdk package rather than a Gemini client. Groq's LPU hardware is built for very low-latency inference, which plausibly helped the team meet the sub-7-second response target in NFR-01; the diagrams should be redrawn to say "Groq API" before final submission so the documentation matches the shipped code.

2.1 Model Selection per Feature

aiService.js does not use one model for everything it picks a model per feature based on task complexity and latency needs:

Function	Model	Temperature	Max Tokens	Why
getExpenseInsight	llama-3.3-70b-versatile	0.5	600	Larger model for higher-quality, multi-section financial writing

autoClassifyTransaction	llama-3.3-70b-versatile	0.1	10	Low temperature for a deterministic single-word category label
askFinancialAssistant	llama-3.1-8b-instant	0.5	300	Smaller/faster model for snappy conversational replies
extractExpenseDetails	llama-3.1-8b-instant	0.1	(JSON mode)	Fast + deterministic structured extraction via response_format: json_object

3. AI Features Implemented

The application implements four AI-powered features, all routed through a dedicated AI Integration Service (services/aiService.js) on the backend:

- AI Financial Report Generator (FR-05, FR-06, FR-09) analyzes the user's transaction history and savings profile and returns one combined report containing a spending-trend insight, personalized budget advice, and a predictive forecast section, generated in a single Groq call and cached as insight_text.
- AI Chat Assistant a conversational assistant that answers natural-language questions about the user's own spending and budget. chatWithAI pre-aggregates the user's income, total spend, and remaining balance server-side before sending that summary to Groq as context, rather than sending raw transaction rows.
- Smart Auto-Categorization (FR-04) automatically suggests a spending category from a transaction's description text as the user types it, restricted to a fixed 10-value enum (Food, Transport, Shopping, Entertainment, Bills, Health, Education, Salary, Investment, Other), visible as "AI Suggested Category" on the Add Transaction screen.
- Smart Auto-fill (extractExpense) parses free-text input (e.g. a pasted receipt line) into structured { amount, category, description } fields using Groq's JSON response mode, though it is not yet wired to a dedicated screen in the current build.

The report generator does not currently write to the separate Recommendations or Forecasts collections shown in the ER diagram all three planned outputs (insight, advice, forecast) are combined into one markdown-formatted text block per call.

3.1 AI Features: Interface, Interaction & Responses

Selected screens from the running application showing the AI features in context:

3.1.1 Smart Auto-Extraction & Transaction Classification

Designed to eliminate the hassle of manual data entry, this feature allows users to input expenses using natural language (e.g., "Spent 500 Taka at KFC"). The AI seamlessly extracts the numerical amount and item description, while automatically classifying the transaction into standard groups (like Food, Transport, Bills, or Shopping). It then converts these extracted details into a JSON object to instantly auto-fill the transaction form, making expense tracking effortless.

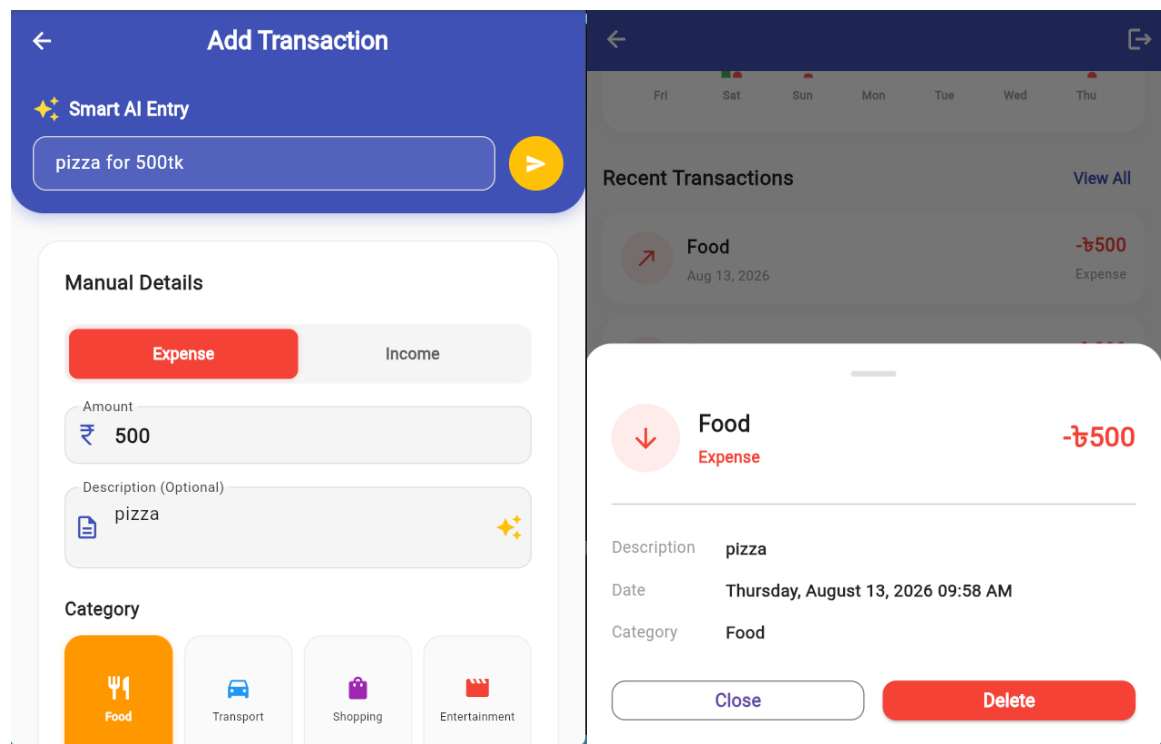


Fig. 1: Smart Auto-Extraction & Transaction Classification

3.1.2 Interactive Financial Assistant Chatbot

This feature allows users to chat directly with an AI regarding their finances. The AI analyzes the user's income, total expenses, and current balance to provide accurate, professional, and well-formatted (Markdown) answers to any financial queries (e.g., "How much did I spend on food this month?").

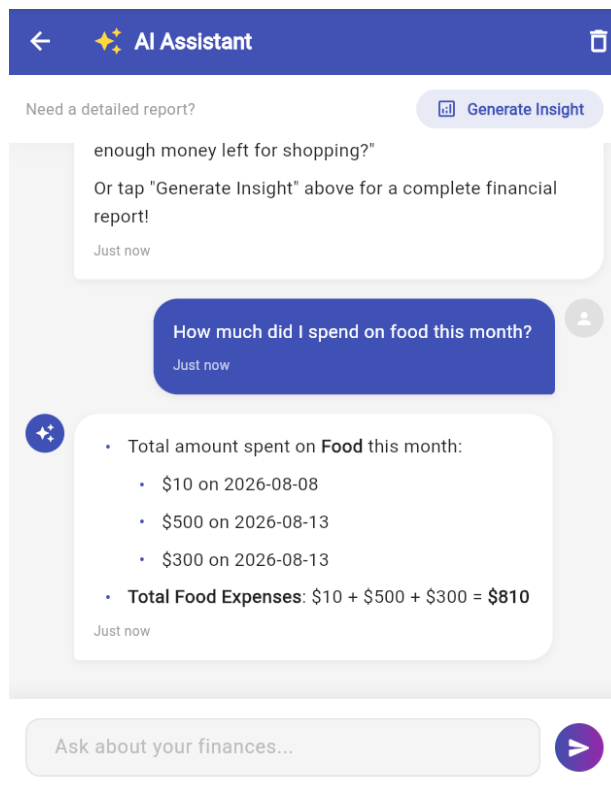


Fig. 2: Interactive Financial Assistant Chatbot

3.1.3 Automated Financial Insight & Report Generator

This is a smart report generator that analyzes the user's complete transaction history and savings goals. With just a single click, it generates a comprehensive financial report featuring three key sections: Spending Trend Insights, Personalized Budget Advice, and Predictive Budget Forecasting.

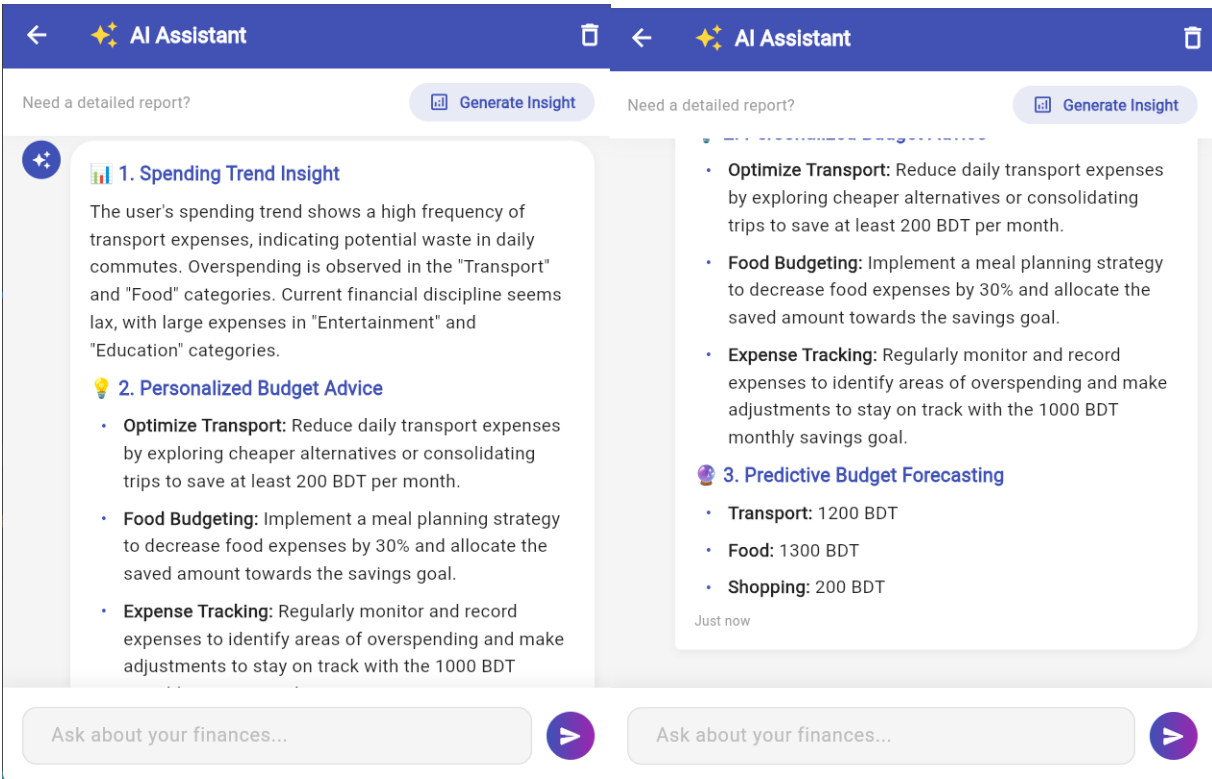


Fig. 3: Automated Financial Insight & Report Generator

4. AI Workflow

The diagram below traces a single AI Report request end to end, from the user opening the app to the AI-generated insight card being displayed:

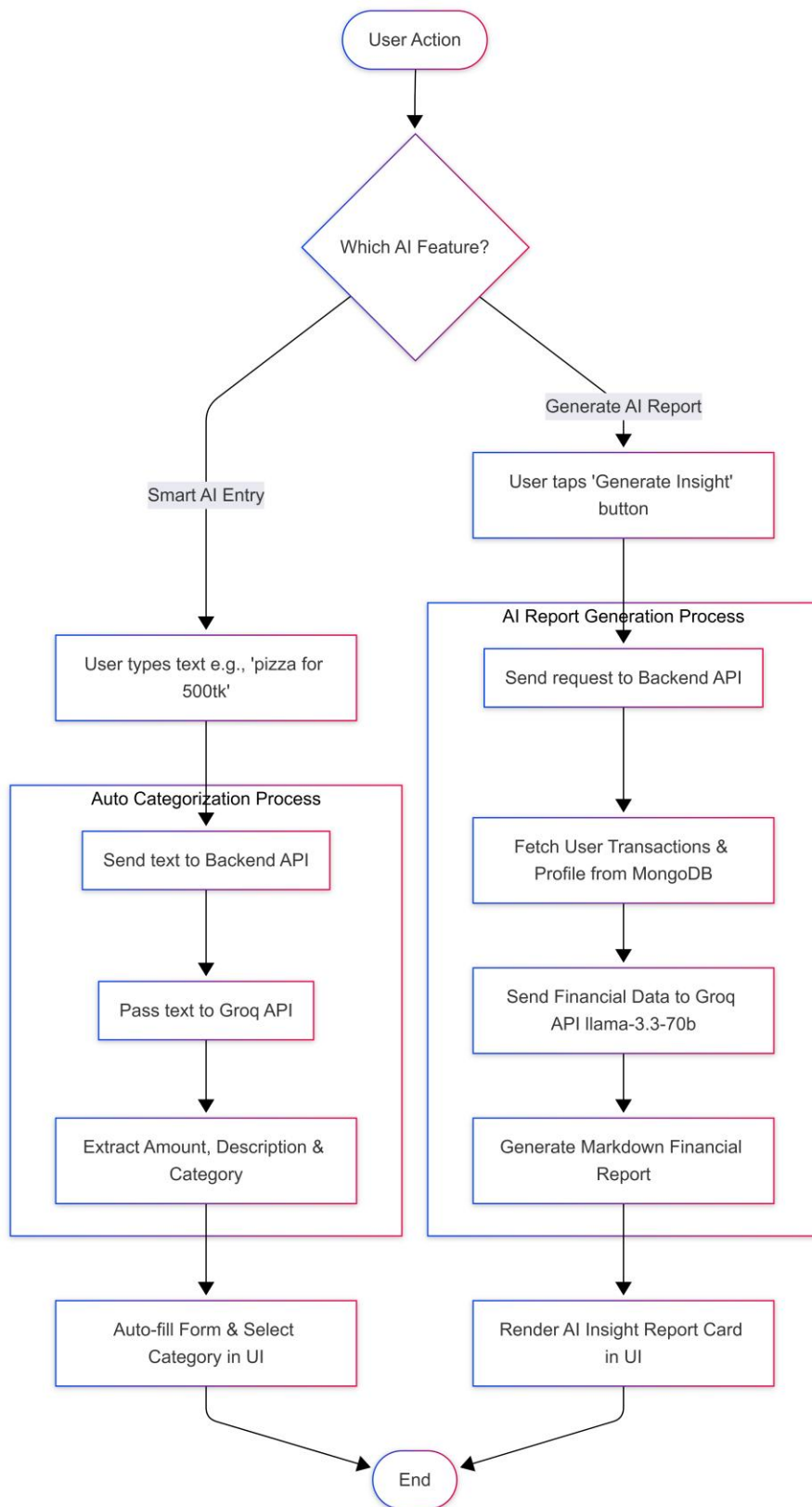


Fig. 4: AI Workflow Diagram for Auto-Categorization and Report Generation

4.1 Workflow Steps

- User Input: The user opens the AI analytics tab from the dashboard.
- Frontend: The Flutter app checks whether the user has transaction history; if none exists, it prompts the user to add more data (the UI copy targets ≥ 1 week of history, per the SRS's detailed use case).
- Backend API: On 'Generate AI Report', `generateInsight()` in `aiController.js` fetches the user's transactions (up to the most recent 50) and their profile (currency, savingsGoal) from MongoDB via Mongoose.
- AI Service: `getExpenseInsight(transactions, userProfile)` in `aiService.js` builds a single structured prompt asking for three sections (spending trend, budget advice, forecast) and calls the Groq API over HTTPS using `llama-3.3-70b-versatile`.
- AI Response: Groq returns one markdown-formatted text block containing all three sections.
- Backend Processing: The controller wraps the text in `{ success: true, insight }` and returns HTTP 200; on failure the try/catch returns HTTP 500. If no valid `GROQ_API_KEY` is configured, `aiService.js` itself returns a friendly fallback string instead of throwing, so the endpoint still responds successfully even with AI unavailable.
- Frontend Display: The returned insight is rendered as an AI insight card in the Flutter UI.

Two things in the diagram below no longer match the shipped code and should be corrected before final submission: (1) it says "Call Google Gemini API", but the live service calls Groq; (2) it shows a "Cache insight in AI_REPORTS collection" step, but the currently-routed `generateInsight` (in `aiController.js`) does not call `AiReport.create` persistence only happens in the unwired `aiReportController.js` version. See Section 8.1 for the evidence behind this second point.

5. Prompt Engineering

The prompts below are reproduced directly from `services/aiService.js` (not idealized examples), so they show exactly what each Groq call receives. All four are single-turn, single-message prompts the "system" role framing shown below is the persona instruction embedded at the top of each user message, since `aiService.js` sends everything as one `{ role: "user" }` message rather than using a separate system role.

5.1 AI Financial Report Generator (getExpenseInsight)

System Prompt

You are an advanced AI Financial Advisor. Provide a structured, user-friendly financial report in English. RULES: No intro/outro text. Use exactly this formatting: 1. Spending Trend Insight ... 2. Personalized Budget Advice (3 bullet points) ... 3. Predictive Budget Forecasting (3 category/amount bullet points).

User Prompt

Analyze this data: [simplified transaction array amount, category, date, type only]. Context: Currency: {currency}, Monthly Savings Goal: {savingsGoal} {currency}.

Expected AI Output

A single markdown-formatted string with three headed sections, e.g. "1. Spending Trend Insight\nThe provided transaction is a single expense of \$20,000 categorized as 'salary'...\n2. Personalized Budget Advice\n1. Re-categorize income: 3. Predictive Budget Forecasting\n..." this is saved verbatim as insight_text (confirmed by the Backend Progress Report's Thunder Client test output).

5.2 Smart Auto-Categorization (autoClassifyTransaction)

System Prompt

(Embedded in the same message, no separate system role)

User Prompt

Classify this expense into exactly ONE category: Food, Transport, Shopping, Entertainment, Bills, Health, Education, Salary, Investment, Other. Description: "Whole Foods Market". Reply ONLY with the category name.

Expected AI Output

"Food" the raw model output is trimmed and checked against the 10-value allow-list; anything not on the list (or any error) is coerced to "Other" before it reaches the client.

5.3 AI Chat Assistant (askFinancialAssistant)

System Prompt

You are a smart, polite financial assistant. Answer the user's question clearly and directly in English. Do not show unnecessary calculation steps. Give a clean, professional answer based on the numbers provided.

User Prompt

User's Financial Summary: Monthly Income: {income}, Total Spent: {totalSpent}, Remaining Balance: {remaining}, Recent Expenses: [up to 10 recent expense items]. User Query: "How much did I spend on food last month?"

Expected AI Output

A short, direct natural-language answer grounded only in the pre-aggregated summary passed in the model never sees raw transaction documents, only the numbers computed server-side in aiController.chatWithAI.

5.4 Smart Auto-fill (extractExpenseDetails)

System Prompt

You are an expert financial assistant. Respond ONLY with a valid JSON object. No extra text or markdown formatting.

User Prompt

Extract expense details from this input: "Lunch at restaurant, 500 taka". Categories allowed: Food, Transport, Shopping, Entertainment, Bills, Health, Education, Salary, Investment, Other.

Expected AI Output

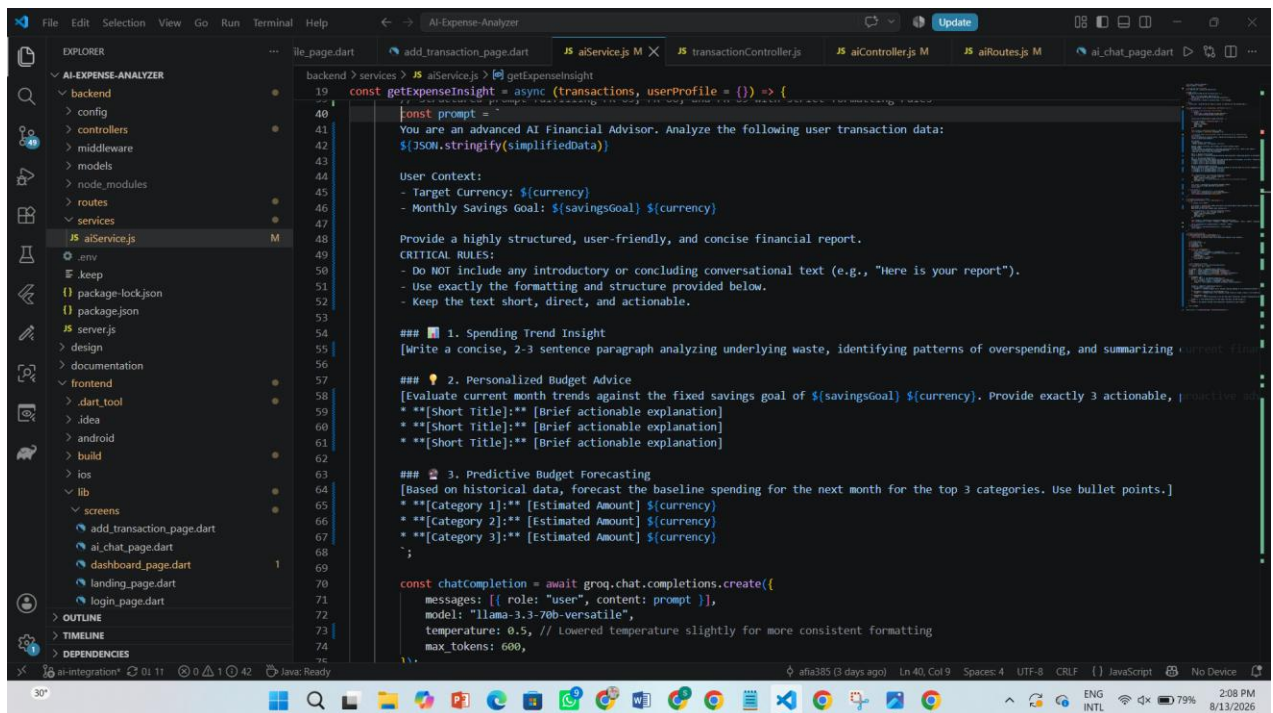
{"amount": 500, "category": "Food", "description": "Lunch at restaurant"} enforced structurally via Groq's response_format: { type: "json_object" } rather than relying on the model to follow formatting instructions unaided.

Design note: three of the four functions (report, classification, extraction) use a low temperature (0.1) or a constrained output format specifically to make results deterministic and easy to validate; only the chat assistant and the report's prose sections use a higher temperature (0.5) where more natural, varied language is actually desirable.

5.5 Prompt Engineering Examples

Feature: Automated Financial Insight & Report Generator

- System Prompt: "You are an advanced AI Financial Advisor. Analyze the following user transaction data: {simplifiedData}. User Context: Target Currency: {currency}, Monthly Savings Goal: {savingsGoal}. Provide a highly structured, user-friendly, and concise financial report."
- User Prompt: *(Since dynamic data is being sent from the backend, the user's real transaction data is passed here dynamically as variables.)*
- Expected AI Output: A Markdown-formatted report with exactly 3 sections: Spending Trend Insight, Personalized Budget Advice, and Predictive Budget Forecasting.



```
backend > services > JS aiService.js > getExpenseInsight
19 const getExpenseInsight = async (transactions, userProfile = {}) => {
40   const prompt = `
41     You are an advanced AI Financial Advisor. Analyze the following user transaction data:
42     ${JSON.stringify(simplifiedData)}
43
44     User Context:
45     - Target Currency: ${currency}
46     - Monthly Savings Goal: ${savingsGoal} ${currency}
47
48     Provide a highly structured, user-friendly, and concise financial report.
49     CRITICAL RULES:
50     - Do NOT include any introductory or concluding conversational text (e.g., "Here is your report").
51     - Use exactly the formatting and structure provided below.
52     - Keep the text short, direct, and actionable.
53
54     ### 1. Spending Trend Insight
55     [Write a concise, 2-3 sentence paragraph analyzing underlying waste, identifying patterns of overspending, and summarizing current financial trends.]
56
57     ### 2. Personalized Budget Advice
58     [Evaluate current month trends against the fixed savings goal of ${savingsGoal} ${currency}. Provide exactly 3 actionable, prioritized budgeting tips.]
59     * **[Short Title]:** [Brief actionable explanation]
60     * **[Short Title]:** [Brief actionable explanation]
61     * **[Short Title]:** [Brief actionable explanation]
62
63     ### 3. Predictive Budget Forecasting
64     [Based on historical data, forecast the baseline spending for the next month for the top 3 categories. Use bullet points.]
65     * **[category 1]:** [Estimated Amount] ${currency}
66     * **[category 2]:** [Estimated Amount] ${currency}
67     * **[category 3]:** [Estimated Amount] ${currency}
68
69     `;
70
71     const chatCompletion = await groq.chat.completions.create({
72       messages: [{ role: "user", content: prompt }],
73       model: "llama-3.3-70b-versatile",
74       temperature: 0.5, // Lowered temperature slightly for more consistent formatting
75       max_tokens: 600,
76     });
77   };
78 }
```

Fig. 5: Prompt Engineering logic for Financial Insights.

5.6 Backend Integration

Dynamic Data Processing

The screenshot demonstrates the backend implementation of the `generateInsight` controller. It highlights how the application dynamically retrieves the authenticated user's real transaction history and profile data (including savings goals and preferred currency) from the database. This contextual data is then seamlessly passed to the AI service, ensuring that the generated financial reports are highly personalized and based on actual user activity rather than static or hardcoded API calls. The logic also includes necessary data validation to handle edge cases, such as preventing the AI call if no transaction history is found.

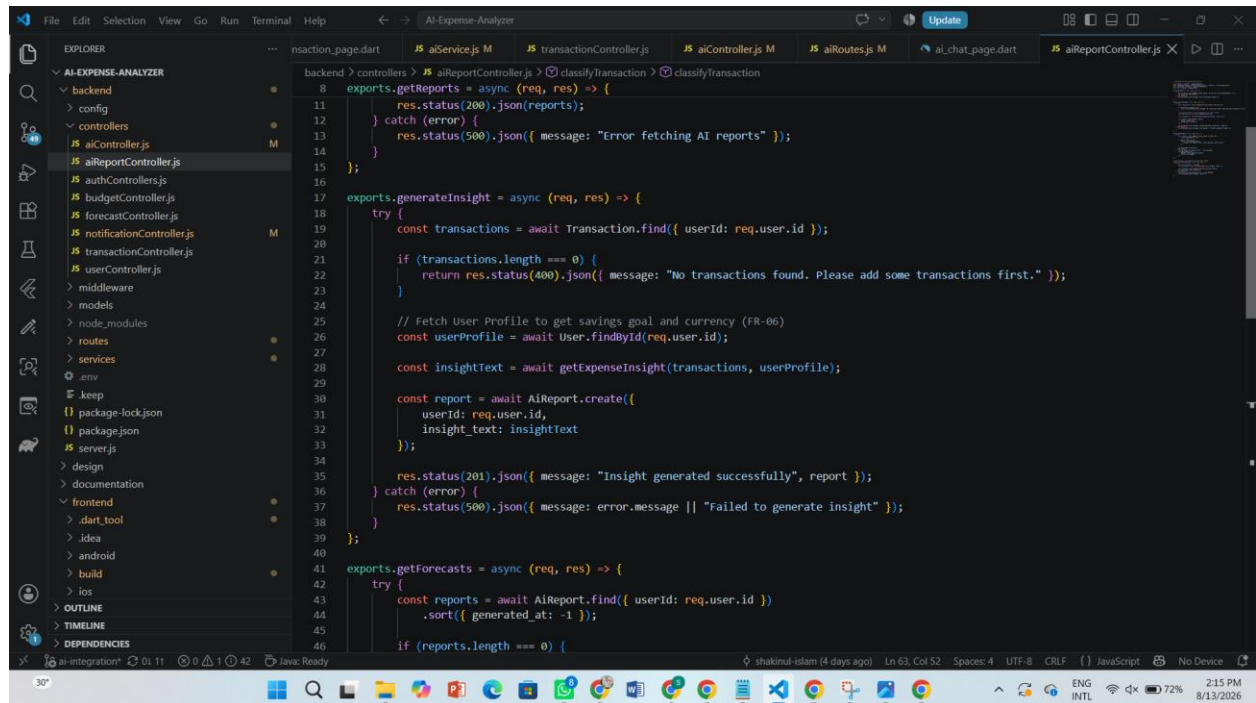


Fig. 6: Backend controller logic demonstrating dynamic data retrieval and AI service integration.

6. AI Response Handling Strategy

Response handling happens at three layers: the AI service itself degrades gracefully when Groq is unavailable, the controllers validate input and catch errors around the service call, and the Flutter `ApiService` (`api_service.dart`) guards the HTTP call. Together they ensure a failed or unconfigured AI integration never crashes the app or blocks core (non-AI) functionality:

- Service-level fallback mode `aiService.js` checks for a valid `GROQ_API_KEY` at startup; if it is missing or malformed, `groq` stays null and every exported function short-circuits to a safe default instead of attempting a network call: `getExpenseInsight` returns "AI is currently running in fallback mode.", `autoClassifyTransaction` returns "Other", `askFinancialAssistant` returns "Sorry, AI assistant is currently unavailable.", and `extractExpenseDetails` returns null. This means the app can run and be demoed even without a configured API key.
- Pre-call validation `generateInsight` rejects the request with HTTP 400 ("No transactions found. Please add some transactions first.") before calling Groq if the user has no transaction history, avoiding a wasted/empty API call.
- Authentication guard `authMiddleware` verifies the JWT on every AI route and returns HTTP 401 for missing or invalid tokens, so AI endpoints are never reachable without a logged-in user.
- Successful responses the controller wraps the Groq result in a JSON envelope (`{success: true, insight/category/answer/data}`) which the Flutter client decodes and renders.
- Server-side errors every function in `aiService.js` has its own try/catch around the Groq call (in addition to the controller's try/catch), so a malformed response or API error is caught close to the source and converted into a fallback string or HTTP 500 rather than an unhandled exception.
- Default fallback values `classifyTransaction` returns `category: "Other"` whenever the description is missing, the returned label isn't in the 10-value allow-list, or the Groq call itself throws.
- Structured-output validation `extractExpenseDetails` uses Groq's `response_format: { type: "json_object" }` so the model is constrained to return parseable JSON, and the surrounding try/catch still guards against a `JSON.parse` failure, returning null → HTTP 400 ("Could not extract data") to the client.
- Network failures on the Flutter side, `ApiService` wraps every AI call in try/catch and reports "Cannot connect to server" rather than throwing an unhandled exception.
- Global backend error handling per the Backend Progress Report, a global Express error-handling middleware maps failure conditions to consistent HTTP codes: 400 (bad/missing input), 401 (missing/invalid/expired JWT), 404 (resource not found), 409 (duplicate email on

registration), and 500 (unexpected DB or Groq failures, logged server-side without crashing the process).

6.1 AI Output Validation

- Pre-call checks (non-empty transaction list) before any AI request is made, avoiding wasted calls and confusing empty results.
- Category allow-list enforcement `classifyTransaction` checks the model's output against the fixed 10-value category list and only accepts an exact match, coercing anything else to "Other".
- Schema-constrained extraction `extractExpenseDetails` uses Groq's native JSON mode rather than asking the model to "please respond in JSON", reducing malformed-output risk.
- Context pre-aggregation for chat rather than sending raw transaction rows to Groq, `chatWithAI` computes income, totalSpent, remaining, and a trimmed list of recent expenses in Node.js first, which both reduces payload size and constrains what the model can "see" or hallucinate about.
- User-facing error messages (`{success: false, message}` or a friendly fallback string) instead of raw exceptions or stack traces reaching the client.

6.2 Backend Testing Evidence

Per the Backend Progress Report, all endpoints listed in Section 3.2 and the wider API surface were exercised with Thunder Client, covering both happy-path requests and deliberate failure cases (duplicate email → 409, invalid password → 401, missing 'amount' field → 400, non-existent transaction ID → 404, missing Authorization header on a protected route → 401). This gives independent, logged evidence that the error-handling behavior described above was actually observed at runtime and not just implemented in theory.

7. Current Development Progress

Frontend and backend integration is complete for authentication (JWT-based, bcrypt-hashed passwords), transactions (including a rule-based proactive budget alert), budgets, and notifications all confirmed working via Thunder Client tests in the Backend Progress Report. The AI Report Generator, AI Chat, Auto-Categorization, and Smart Auto-fill features are integrated end-to-end

with the Groq API. Predictive Forecasting is the least complete AI feature: the Forecast schema and a read endpoint exist, but the report generator does not currently populate the Forecast (or Recommendations) collection forecast content currently only exists as prose inside the combined AI report text.

7.1 Limitations

- Route/controller mismatch (evidenced, not assumed) aiRoutes.js currently wires /api/ai/generate and /api/ai/reports to aiController.js, whose generateInsight does not persist to AiReport and whose getReports is a hard-coded stub returning []. But the Backend Progress Report's own Thunder Client screenshots for these exact endpoints show responses shaped like { message: "Insight generated", report: { _id, insight_text, generated_at } } and a real array of saved reports which match aiReportController.js, not aiController.js. Either the routes file changed after testing or a different version was live at test time; the team should confirm which controller is authoritative and remove the other before final submission.
- Forecast/Recommendations gap the single combined-report design (Section 3.1) means the Forecast and Recommendations collections are never written by any AI call in the reviewed code, so forecastController.getForecasts has no live data source. The Backend Progress Report's own "Forecast:" test screenshot appears to reuse the /api/budgets response rather than an actual /api/forecasts call, which is consistent with this gap.
- Rule-based vs AI-driven alerts the proactive budget notifications in transactionController.addTransaction ("Budget Exceeded", "Proactive Budget Warning") are deterministic percentage-threshold checks, not Groq generated text; only the AI Report, Chat, Classification, and Extraction features call the LLM. This distinction matters for the viva.
- Documented-but-unimplemented endpoints the SRS/System Design and Backend Progress Report list PUT /budgets/:id, PUT /users/change-password, GET /transactions/:id, and PUT /transactions/:id, but no corresponding route or controller function exists in the files reviewed for this report (budgetController.js has no update function; userController.js has no change-password function; transactionRoutes.js has no :id GET/PUT routes). These should be implemented or removed from the documentation for consistency.

- Diagram/code drift the Activity and Architecture diagrams both label the AI layer "Google Gemini API"; the shipped code calls Groq. The diagrams should be redrawn before final submission.
- The one-week-of-history precondition described in the SRS's use case and activity diagram is not enforced in generateInsight, which only checks for a non-empty transaction list.
- Category suggestions are based on description text only and may misclassify ambiguous merchant names, and the fixed 10-category list (Food, Transport, Shopping, Entertainment, Bills, Health, Education, Salary, Investment, Other) differs from the illustrative categories shown in early UI mockups (e.g. "Groceries", "Dining").
- AI responses depend on Groq API availability and quota; heavy usage may hit rate limits during grading/demo, though the built-in fallback mode (Section 7) means the app stays functional even if the key is revoked or missing.

7.2 Future Improvements

- Resolve the aiController.js / aiReportController.js discrepancy and delete the unused controller.
- Implement forecast and recommendation generation as their own Groq calls that populate the Forecast and Recommendations collections, completing FR-09 end-to-end.
- Implement the documented-but-missing endpoints (budget update, password change, transaction get/update by ID) or remove them from the SRS/System Design if out of scope.
- Redraw the Activity and Architecture diagrams to say "Groq API" instead of "Google Gemini API".
- Add few-shot examples to the categorization prompt to improve accuracy on ambiguous merchant names.
- Enforce the ≥ 1 -week-of-history precondition server-side, matching the documented use case.

8. GitHub Repository

Repository Link: <https://github.com/shakinul-islam/AI-Expense-Analyzer>

Repository Structure & Version Control The GitHub repository is organized strictly according to the recommended structure, featuring dedicated directories for frontend, backend, documentation, and a newly added “ai” folder containing prompts and sample payloads. The

commit history demonstrates regular, incremental updates with meaningful commit messages (e.g., "Added ai directory with prompts...", "Integrated AI and related update"), reflecting a proper and well-maintained version control workflow throughout the AI integration phase.

The screenshot shows the GitHub interface for the repository 'AI-Expense-Analyzer' by user 'afia385'. The repository is public and has 2 branches and 0 tags. The commit history shows a series of incremental updates, including the addition of an 'ai' directory, integration of AI features, and updates to the project overview and directory names. The file structure includes directories for 'ai', 'backend', 'design', 'documentation', and 'frontend', as well as files for '.gitignore' and 'README.md'.

Commit Message	Commit Hash	Time Ago	Commits
Added ai directory with prompts and sample payloads for all AI features	47f4221	5 hours ago	27 Commits
Added ai directory with prompts and sample payloads for all ...		5 hours ago	
Integrated AI and related update		4 days ago	
setup for week5		2 months ago	
setup for week5		2 months ago	
Integrated AI and related update		4 days ago	
Updated .gitignore to include backend and environment files		2 months ago	
Revise project overview and update directory names		2 months ago	

Fig. 7: GitHub repository structure and recent commit history demonstrating proper AI integration and version control.